

Network Operations Automation at DE-CIX

Lucas Immanuel Nickel

AUTOCON S

THE NETWORK AUTOMATION CONFERENCE

Servus, I'm Lucas.

- Building and operating NetOps automation
- Focused on making operational automation reliable, observable, and maintainable



Lucas Immanuel Nickel
Network Operations Engineer

At DE-CIX scale, automation is not optional

The scale of DE-CIX

- Global interconnection platform
- 3,700+ connected networks
- 79 EB exchanged in 2025
- 100+ routers and hundreds of circuits worldwide

Automation at scale needs guardrails

- Automation is required at scale
- Automation must be trustworthy, repeatable, and maintainable
- Otherwise, automation becomes the root cause of the next incident with your name in the Git log

At this scale, NetOps automation needs SRE practices:
reliable delivery, observability, and safe failure.

Operational automation, not network orchestration

Network orchestration

- Provisioning
 - Configuration rollout
 - Lifecycle management
- Owned by the **NORC** team

Operational automation

- Monitoring & Alarming
 - Data correlation
 - Workflow automation
- Owned by the **NetOps** team

For more on **NORC**, I recommend:

DENOG17 - **Inventing the wheel** - **Network Orchestration at scale**

Our problem is not writing code

15+

production cron jobs

20+

operational scripts

3

production web services



One library
to rule them all

500+

executions per day

Useful automation starts with operational pain



Repeated
toil



Missing
visibility



Friction between
systems and work

Where it hurts:
maintenance handling, software upgrades, route servers, ...

When do we automate?

Repeated toil

→ daily or weekly work

Scale pressure

→ tens of routers, hundreds of devices, hundreds of tickets

Good data

→ known systems, APIs, source-of-truth available

Manageable blast radius

→ failure can be constrained, reviewed, or rolled back

Automate when the work is repetitive,
data-backed, and the blast radius is understood.

From legacy hosts to Kubernetes

Legacy

- Shared hosts
- Fragile deployments
- No standardized path for deploying web services
- Limited workload isolation, visibility, and rollback

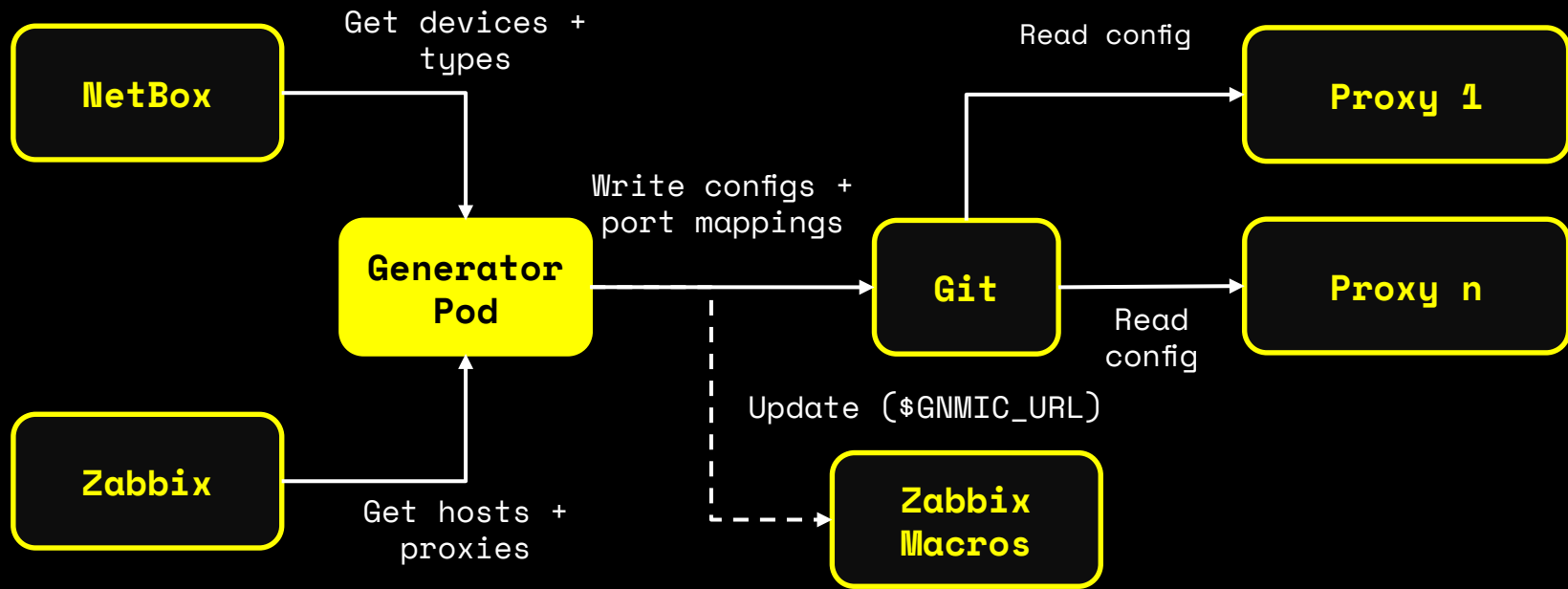
Today

- Cron jobs and web services run in Kubernetes
- Standardized CI/CD delivery
- Isolated workloads with better visibility and rollback support

GitOps-style workload delivery

1. Engineers start with a `main.py` in a `UV` project
2. One `shared library` to abstract boilerplate and glue code
3. They define the workload in a `Helm chart`
4. They push the change to `Git`
5. `Review` and `CI/CD` validate the change
6. `Kaniko` builds, `Harbor` stores, `Helm` deploys, `Kubernetes` runs
7. `Renovate` keeps dependencies up to date

gNMIC config generation



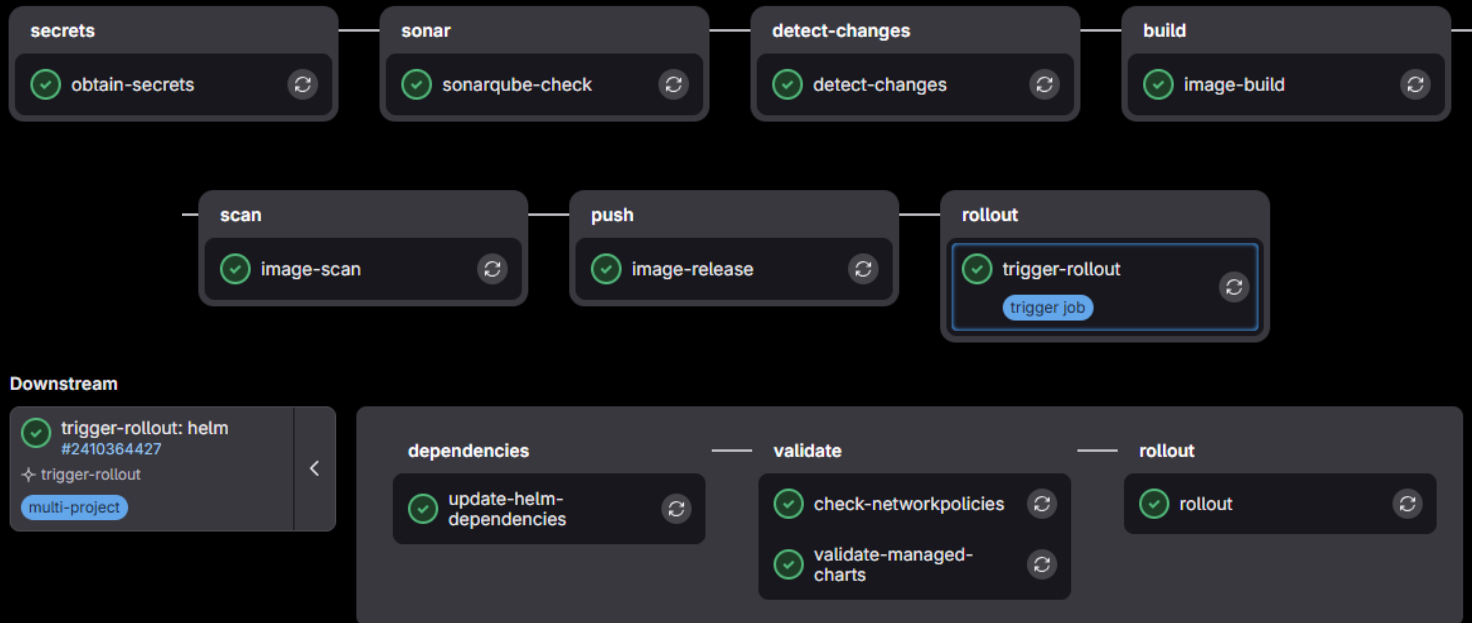
As part of modernizing monitoring, we generate gNMIC proxy configs to expose device telemetry as Prometheus metrics.

Define how it runs

```
gnmic-index-targets

- name: gnmic-index-targets
  schedule: '*/*30 * * * *'
  startingDeadlineSeconds: 900
  owner: lnickel
  ttlSecondsAfterFinished: 120
  maxExecutionTimeSeconds: 120
  maxFailures: 1
  connectsTo:
    target-a: true
    target-b: false
    [...]
  image:
    repository: redacted
    tag: redacted
  secrets:
    - [...]
  env:
    - [...]
  resources:
    [...]
```

Deliver it like every other workload



Monitor it by failure mode, not by tool

Delivery

- Did the change reach production?
- Who gets notified when it doesn't?

Cronjobs

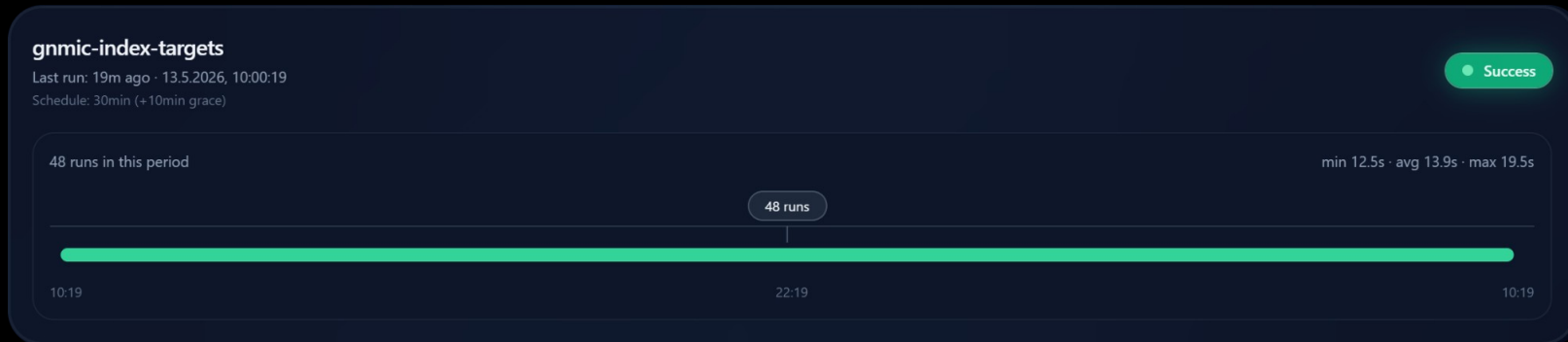
- Can you even scrape it fast enough?
- Where do operators see failed runs?

Scripts

- What happened during execution?
- How to keep the operator in the loop?

Services

- Is it healthy and useful?
- Is stale data becoming a problem?



Failure moved left

1 in 3

deploys got rejected before
production in April

0

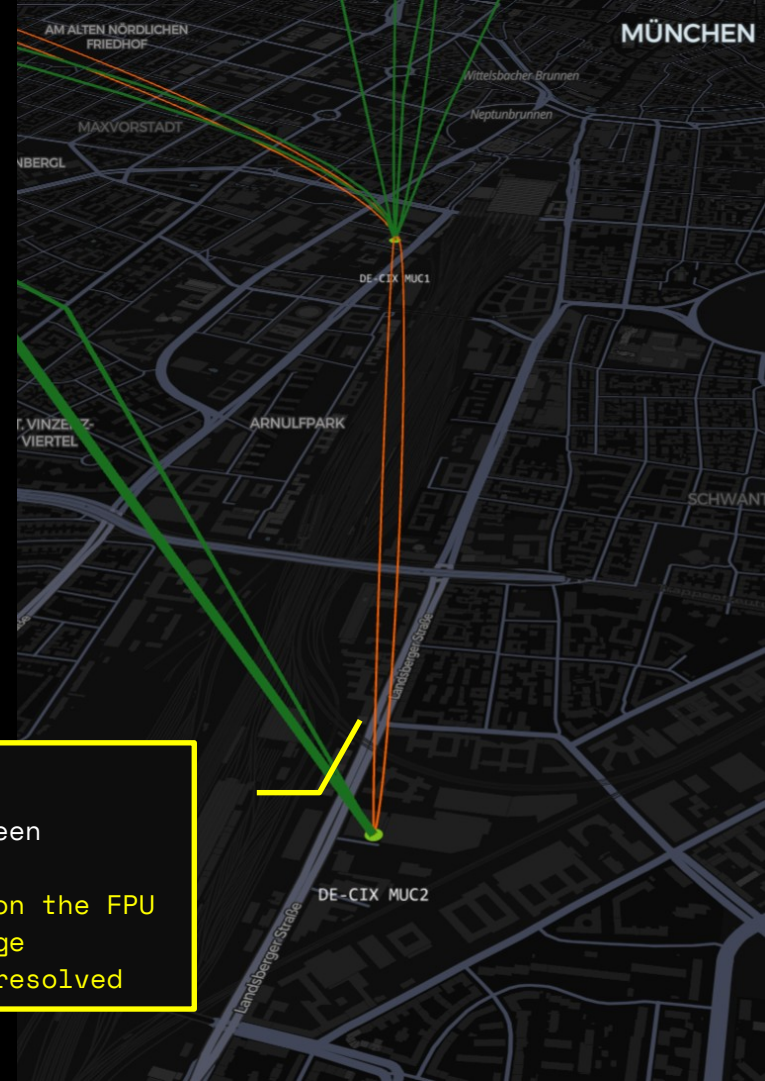
deploys failed on the production
path in April

Geo

- Provides a **geographic view** of the interconnection platform
- Correlates **NetBox** and **Zabbix data**
- Resolves **sites**, **circuits**, and **LAGs**
- Handles **imperfect data**
- API is used by other consumers
- Used across the company

FPU-to-FPU case

- A **FPU** switches traffic between primary and backup fibers
- **NetBox** objects **terminates** on the FPU
- **Zabbix** data sits on the **edge**
- Geo leaves the endpoint **unresolved**



Geo is a data problem

- NetBox and Zabbix are **not always complete**
- Traces can be **partial, missing, or wrong**
- Names **do not always follow conventions**
- Relationships must be **inferred**
- **Not every object** belongs on the map



Production automation must degrade safely



Validate and normalize
inputs before action



Surface failures where
operators can see them



Prefer incomplete
but honest output



Automate defensively

Patterns that survived production

Delivery

- One release path for every workload
- Let development fail before production
- Separate business logic from platform concerns

Data

- Treat data as untrusted input
- Make generated output deterministic and auditable
- Prefer visible partial failure over silent success

Automation is
production
infrastructure.
Treat it like it.



[Lucas Immanuel Nickel](#)
Network Operations Engineer
@ DE-CIX



Questions?